

Rapport de conception

Étape 3 — Conception visuelle et intégration front-end / back-end

PROJET livresgourmands.net

COURS Programmation Web avancée

ENSEIGNANTE Kahina Tamazouzt

MEMBRES Khalid Asseffar et Youness Bellouk

SESSION Hiver 2026

Ce dossier présente les choix de conception visuelle, l'organisation du frontend React, ainsi que l'intégration avec l'API REST Node.js / Express / MySQL développée lors de l'étape précédente.

1. Introduction

Le projet livresgourmands.net est une plateforme e-commerce dédiée à la vente de livres de cuisine. Après avoir conçu les diagrammes UML lors de la première étape, puis développé une API REST sécurisée en Node.js, Express et MySQL lors de la deuxième, cette troisième étape consiste à donner au projet une véritable interface utilisateur. Nous passons donc d'une API testable uniquement avec Postman à une plateforme web complète, avec un frontend React connecté à notre backend.

2. Objectif de l'étape 3

L'objectif principal était de concevoir une expérience utilisateur claire, moderne et fonctionnelle. Concrètement, nous devons réaliser des maquettes visuelles, développer un frontend React, le connecter à notre API REST via Axios, gérer un panier dynamique avec persistance, et offrir un espace administrateur permettant de gérer le catalogue. L'idée n'est pas seulement de "brancher" l'interface au backend, mais de proposer un site qui ressemble vraiment à une librairie en ligne professionnelle.

3. Identité visuelle

Nous avons choisi un style premium qui évoque une librairie haut de gamme spécialisée en gastronomie. L'identité repose sur trois couleurs : un vert forêt profond comme couleur principale, un doré chaud comme accent, et un fond crème presque blanc pour aérer l'ensemble. Ces trois teintes proviennent directement du logo, qui représente un livre ouvert surmonté d'une toque, d'une fourchette et d'une cuillère.

La typographie associe une serif élégante (Cormorant Garamond) pour les titres et une sans serif lisible (Inter) pour le texte courant. Les cartes ont des coins arrondis, des ombres très douces, et les boutons utilisent des dégradés discrets. L'objectif est de donner une impression luxueuse sans nuire à la lisibilité.

4. Maquettes Figma

Les maquettes ont été réalisées dans Figma à partir du frontend final, afin de visualiser la mise en page de chaque écran avant et après le développement. Nous avons préparé les vues suivantes :

- Accueil desktop et mobile
- Catalogue / Ouvrages desktop et mobile
- Détail d'un ouvrage
- Panier desktop et mobile
- Connexion et inscription
- Dashboard administrateur
- Gestion des ouvrages (CRUD)

Ces maquettes nous ont aidés à valider la hiérarchie des informations, l'espacement entre les sections et le comportement responsive. La capture ci-dessous présente une vue d'ensemble du board Figma regroupant les principaux écrans.

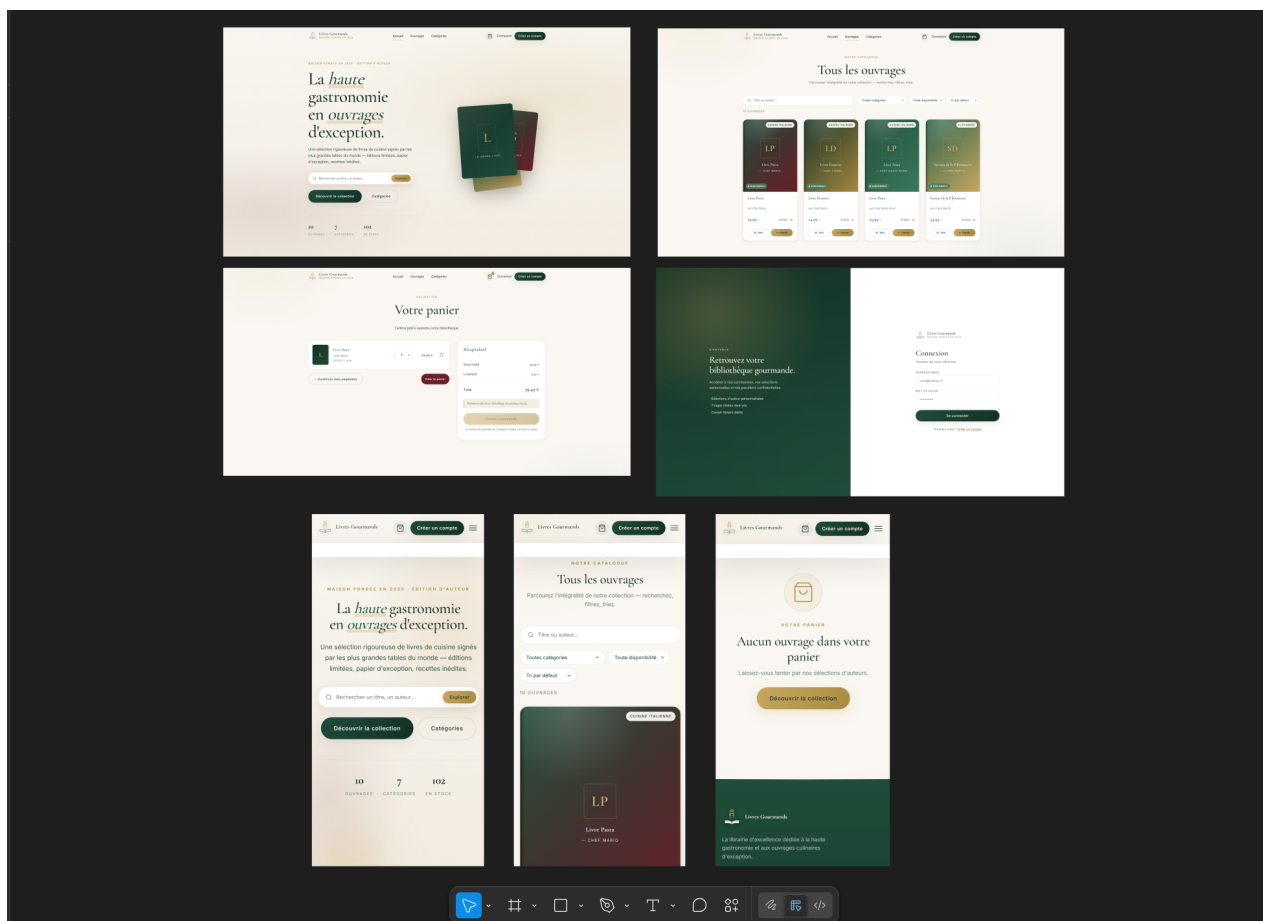


Figure 1 — Vue d'ensemble du board Figma (desktop + mobile)

5. Parcours utilisateur

Trois parcours ont guidé la conception. Le premier est celui du visiteur curieux : il arrive sur la page d'accueil, découvre une sélection d'ouvrages, parcourt les catégories et consulte une fiche produit. Le second parcours est celui du client : il ajoute un ouvrage au panier, modifie la quantité, consulte le total, puis se connecte ou crée un compte. Enfin, l'administrateur peut se connecter à un dashboard dédié et gérer les ouvrages et catégories en temps réel.

Nous n'avons pas réalisé de prototype Adobe XD séparé. Le frontend React étant fonctionnel et connecté à la base de données, il permet de démontrer directement toutes les interactions lors de la vidéo de démonstration. Cette approche nous a semblé plus représentative du projet réel qu'un prototype statique.

6. Structure du frontend React

Le frontend est organisé en dossiers clairs, chacun avec une responsabilité précise :

- components/ — éléments réutilisables (Navbar, Footer, BookCard, Logo, ProtectedRoute)
- pages/ — une page par route, avec son fichier CSS dédié
- context/ — AuthContext (utilisateur connecté) et CartContext (panier)
- services/ — api.js, instance Axios partagée avec intercepteur JWT
- layouts/ — MainLayout (site public) et AdminLayout (dashboard)
- assets/ — logo et ressources statiques

Cette séparation rend le projet plus simple à maintenir et permet de retrouver rapidement où se trouve chaque morceau de logique.

7. Pages développées

- Accueil : section hero animée, sélection d'ouvrages, catégories, bloc éditorial.
- Catalogue (Ouvrages) : liste complète avec recherche par titre ou auteur, filtre par catégorie et tri.
- Détail d'un ouvrage : informations complètes, gestion de la quantité, ajout au panier.
- Panier : items dynamiques, modification des quantités, total recalculé en direct.
- Connexion et inscription : formulaires reliés à l'API d'authentification.
- Dashboard administrateur : statistiques globales, accès rapide aux modules de gestion.
- Gestion CRUD des ouvrages et des catégories : tableau, drawer latéral pour création / édition, suppression.

8. Intégration avec le backend

Le frontend communique avec l'API Node.js / Express via une instance Axios configurée dans `services/api.js`. La `baseUrl` pointe vers `http://localhost:3000/api`. Un intercepteur attache automatiquement le token JWT lorsqu'il est présent dans le `localStorage`, ce qui évite de répéter ce code à chaque appel.

Endpoints principaux utilisés :

- GET `/api/ouvrages` — liste des ouvrages (avec nom de catégorie joint)
- GET `/api/ouvrages/:id` — détail d'un ouvrage
- GET `/api/categories` — liste des catégories
- POST `/api/auth/login` — connexion et récupération du JWT
- POST `/api/auth/register` — création de compte
- POST / PUT / DELETE `/api/ouvrages` — administration du catalogue (rôle admin)
- POST / PUT / DELETE `/api/categories` — administration des catégories (rôle admin)

Les données affichées ne sont jamais codées en dur : elles viennent toutes de la base MySQL via l'API. Si le backend n'est pas démarré, le frontend affiche un message d'erreur clair au lieu de planter.

9. Gestion du panier

Le panier est entièrement géré côté frontend grâce à la Context API de React. Un `CartContext` expose les fonctions `addToCart`, `removeFromCart`, `updateQuantity` et `clearCart`, ainsi qu'un total et un compteur d'articles calculés automatiquement. À chaque modification, l'état du panier est sauvegardé dans le `localStorage` du navigateur. L'utilisateur peut donc fermer l'onglet et revenir : son panier est toujours là.

Cette approche évite de surcharger le backend tant que l'utilisateur n'a pas validé sa commande, tout en offrant une expérience fluide.

10. Authentification et espace administrateur

L'authentification repose sur des tokens JWT générés par le backend lors du login. Le frontend stocke le token dans le `localStorage`, puis le réutilise pour chaque requête nécessitant une autorisation. Un `AuthContext` suit en permanence l'utilisateur connecté et son rôle.

Les routes administrateur (`/admin`, `/admin/ouvrages`, `/admin/categories`) sont protégées par un composant `ProtectedRoute`. Si l'utilisateur n'est pas connecté, il est redirigé vers la page de connexion ; s'il n'a pas le rôle administrateur, il est redirigé vers l'accueil. Le dashboard offre une vue d'ensemble avec des cartes statistiques (nombre d'ouvrages, catégories, stock total) et des tableaux CRUD complets pour gérer le catalogue.

11. Responsive design et accessibilité

L'ensemble du site a été pensé pour s'adapter aussi bien aux écrans desktop qu'aux téléphones. Les grilles passent automatiquement de plusieurs colonnes à une seule, le menu de navigation se transforme en menu burger en dessous de 960 pixels, et les images conservent leurs proportions. Côté

accessibilité, nous avons veillé à des contrastes suffisants entre texte et fond, à des boutons clairement identifiables, à des labels pour chaque champ de formulaire, et à des aria-label sur les icônes interactives comme le panier.

12. Limites et améliorations possibles

Le projet reste un travail scolaire, et plusieurs points pourraient encore être améliorés :

- Adobe XD n'a pas été utilisé comme prototype séparé — le frontend fonctionnel sert de prototype interactif.
- Le paiement n'est pas réellement implémenté : le bouton "Passer commande" est volontairement désactivé.
- La gestion complète des commandes côté administrateur reste à développer.
- Les avis et commentaires sur les ouvrages sont prévus dans la base de données mais pas encore exposés côté frontend.
- Le filtrage pourrait être enrichi (fourchette de prix, plusieurs catégories à la fois).

13. Conclusion

L'étape 3 nous a permis de relier toutes les pièces du projet : la modélisation faite à l'étape 1, l'API sécurisée développée à l'étape 2, et maintenant une interface utilisateur React moderne et responsive. Nous sommes passés d'un projet purement technique à une vraie expérience visuelle, proche de ce qu'on retrouve sur une plateforme e-commerce réelle.

Au-delà du code, cette étape nous a surtout fait travailler la cohérence entre design, ergonomie et intégration : choisir des couleurs en accord avec le logo, organiser les composants React de manière maintenable, gérer correctement l'état global avec la Context API et les bonnes pratiques de sécurité côté token. Le résultat est une base solide, prête à évoluer.

